Module 3 — Customization

Lesson 3.2

settings.json — permissions, env, and scope

settings.json configures permissions, env vars, hooks, and MCP servers. Three scopes (user, project, local) compose; understand the precedence.

Why it matters

`CLAUDE.md` tells Claude *what* the project is. `settings.json` tells Claude Code *what it's allowed to do*. The two together turn a generic install into a per-project workflow.

The three scopes

File	Scope	Committed?	Use for
`~/.claude/settings.json`	User-global	No	Your personal defaults (model, theme, your shared permissions)
`<project>/.claude/settings.json`	Project (shared)	Yes	Team-wide permissions, hooks, MCP, env defaults
`<project>/.claude/settings.local.json`	Project (private)	No (gitignored)	Personal overrides for this project

Precedence (later wins): user → project → local.

What goes in settings.json

The big categories:

```
{
  "permissions": {
    "allow": ["Bash(pnpm test:*)", "Bash(git status)", "Read", "Grep"],
    "ask":   ["Bash(git push)"],
    "deny":  ["Bash(rm -rf:*)"]
  },
  "env": {
    "NODE_ENV": "development",
    "PYTHONDONTWRITEBYTECODE": "1"
  },
  "hooks": {
    "PreToolUse": [...],
    "PostToolUse": [...]
  },
  "mcpServers": {
    "linear": { "command": "npx", "args": ["@linear/mcp"] }
  },
  "model": "claude-opus-4-7"
}
```

(Exact key names and shapes evolve. Check `claude --help` and the docs against your installed version.)

Permissions in depth

Three lists: `allow`, `ask`, `deny`. Most specific match wins; on tie, `deny` beats `ask` beats `allow`.

Tool patterns

- `"Read"` — allow the Read tool unconditionally

- `"Bash(pnpm test:*)"` — allow Bash commands matching `pnpm test:*` (the `:*` is wildcard)
- `"Bash(git status)"` — allow exactly `git status`
- `"Edit(src/**)"` — allow edits to files matching the glob

Be specific. A blanket `"Bash"` allow is essentially `--dangerously-skip-permissions` for the shell.

Recommended baseline for a typical project

```
{
  "permissions": {
    "allow": [
      "Read", "Glob", "Grep",
      "Bash(git status)", "Bash(git diff:*)", "Bash(git log:*)",
      "Bash(pnpm install)", "Bash(pnpm test:*)", "Bash(pnpm typecheck)",
      "Bash(pnpm lint:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)",
      "Bash(curl:*)"
    ]
  }
}
```

This allows read-only inspection and your build tooling without prompts, but blocks destructive shell calls.

The `fewer-permission-prompts` skill

Built-in skill that scans your transcripts and proposes an allowlist tuned to what you actually use. Run it after a week of normal use:

```
> /fewer-permission-prompts
```

Best ROI single command in the customization module.

env

For values that should be set in every Claude session for this project:

```
{
  "env": {
    "DATABASE_URL": "postgres://localhost/myapp_dev",
    "OTEL_SDK_DISABLED": "true"
  }
}
```

Don't put secrets here if the file is committed — use `settings.local.json` (gitignored) for personal secrets, or pull from your shell environment.

Local overrides

`settings.local.json` is for things you don't want to share with the team:

- Personal API keys for MCP servers

- Permission allowlist additions for your workflow that aren't team consensus
- A different model than the team default

The file is auto-gitignored by Claude Code if it created your `.claude/` directory.

The /config and /permissions commands

You don't have to hand-edit JSON. Two slash commands cover most cases:

- `/config` — interactive UI for global settings (theme, model, etc.)
- `/permissions` — view and edit the resolved permissions list

For hooks and MCP, hand-editing is still the norm.

Try it

1. Open or create `.claude/settings.json` in a project.
2. Run `/permissions` to see what's currently allowed.
3. Run `/fewer-permission-prompts` and apply its suggestions.
4. Add a `deny` rule for `Bash(rm -rf:*)`. Confirm with `/permissions`.
5. Commit `settings.json`. Confirm `settings.local.json` is gitignored if it exists.

Gotchas

- **Wildcard syntax matters.** `"Bash(pnpm test*)"` is NOT the same as `"Bash(pnpm test:*)"`. The colon-star is the documented form.
- **Local overrides shared rules silently.** If a team member can't reproduce your behavior, check `.local.json`.
- **`env` doesn't replace your shell env**, it adds to it. Conflicts go to whichever loads last.
- **MCP config changes require restart.** Edits to `mcpServers` don't take effect until the next `claude` invocation.

Further reading

- 3.3 Hooks — the `hooks` section in depth
- 5.2 Connecting MCP servers — the `mcpServers` section
- Official settings docs: https://docs.claude.com/en/docs/claude-code/settings

Next

→ 3.3 Hooks — the automation layer